The Mathematics of tagd

A Formal Description of the Mathematical Structures
in the tagd Semantic-Relational System

Isaac N Colley

September 2026

"To be, is to be related."
— tagd Maxim

Abstract

The tagd system was built from a vision of a hierarchical structure of knowledge that is fully connected and consistent within itself. This paper makes the mathematics of that vision explicit. Its central thesis is that *information is structural*: in tagd, meaning is defined through relations. A tag is situated by its subordinate relation and described by explicit and inherited predicates. Type declarations extend inheritance without relocating tags. Rank strings encode this hierarchy as a partial order, whose Alexandrov topology gives precise form to containment, neighbourhoods, and continuity. The paper develops these connections and explores how they can support a relational account of computation.

Contents

1 Primitives **2**
- 1.1 Tags . . . 2
- 1.2 The Root and the Axiom . . . 2

2 The Subordinate Relation and the Rooted Tree **3**
- 2.1 Subordinate Relations . . . 3
- 2.2 The Tagspace as a Rooted Tree . . . 3

3 Rank: Encoding the Tree in Strings **3**
- 3.1 The Rank Function . . . 3
- 3.2 The Prefix Partial Order . . . 4
- 3.3 Verifying the Partial Order Axioms . . . 4
- 3.4 Subtrees and Their Intersection . . . 4
- 3.5 Lexicographic Total Order . . . 5
- 3.6 CS Tree versus Topological Poset: Orientation . . . 5

4 The Tagspace Topology **5**
- 4.1 From Partial Order to Topology . . . 5
- 4.2 Verifying the Topology Axioms . . . 5
- 4.3 Specialization Order . . . 5
- 4.4 Continuity as Order-Preservation . . . 6

5 The Hard Tag Base: A Common Subspace **6**
- 5.1 Hard Tags . . . 6
- 5.2 The Hard Tag Subspace . . . 6
- 5.3 Interoperability via the Common Base . . . 6

6 Predicates: Fibered Structure **7**
6.1 Relators Are Tags . . . 7
6.2 Predicates as Horizontal Relations . . . 7
6.3 Declared Predicate Fibers . . . 7

7 Inheritance **7**
7.1 Query Semantics as Selection of Sets . . . 8

8 Structural Identity **9**
8.1 Identity Is Position . . . 9
8.2 The Canonical Form . . . 9

9 Deeper Structures **9**
9.1 The Adjunction Between Order and Topology . . . 9
9.2 Topological Properties of the Tagspace . . . 10
9.3 Modal Operators: Interior and Closure . . . 10
9.4 Constructive Existence: Tags as Witnesses . . . 11

10 Directions for Formalization **11**
10.1 Kripke Semantics and Modal Queries . . . 11
10.2 Martin-Löf Type Theory and Dependent Structure . . . 11
10.3 Computation and Processes . . . 11
10.4 Signed Merkle Versions of Tagspaces . . . 11

A Implementation Correspondences **12**

1 Primitives

1.1 Tags

A **tag** is the atomic unit of the tagd system: a node in the hierarchy and a point in the tagspace topology developed below. Tag identifiers are UTF-8 labels [9]. A tag carries two distinct identities:

- **Nominal identity**: a UTF-8 label unique within a tagspace.
- **Structural identity**: its position in the tagspace hierarchy, given by its rank, defined by its subordinate relation (aka identity relation) [7].

Definition 1.1 (Tag Set)**.** *Let T be the set of all tags in a tagspace. Each tag $t \in T$ is identified by a string* $\mathrm{id}(t) \in \Sigma^*$*, where Σ^* denotes the set of all finite UTF-8 strings.*

In `dog _is_a mammal`, the label supplies nominal identity and the subordinate relation supplies structural identity.

1.2 The Root and the Axiom

Every tagspace has a distinguished root tag: `_entity`.

Definition 1.2 (Axiomatic Root)**.** *The tag* ***`_entity`*** *is the* ***axiomatic root*** *of every tagspace. It is the unique fixed point of the parent function extended to the root: the one tag whose super-object is itself.*

TAGL writes the root axiom as `_entity _sub _entity`; the symbol `<:` denotes `_sub`.

In hierarchical terms, `_entity` is the **root** of the tagspace — the tag that is an ancestor of everything, including itself.

2 The Subordinate Relation and the Rooted Tree

2.1 Subordinate Relations

A tag's structural identity is defined by one subordinate (sub) relation: a sub-relator and a parent [7, 8]. The code represents these as `sub_relator` and `super_object`.

Structural relators and type relators occupy distinct families in the hard-tag base.

Remark 2.1 (Direct subordination versus containment)**.** *A subordinate relation records the pair $(s, p(t))$, where s is its sub-relator and $p(t)$ its immediate parent. This edge is distinct from the reflexive, transitive ancestor relation encoded by rank prefixes. For a non-root tag, the parent is its unique lower cover in the ancestor-first order used below; the root has no lower cover.*

2.2 The Tagspace as a Rooted Tree

The subordinate relation organizes tags into a **rooted tree**.

Definition 2.1 (Tagspace Tree)**.** *A tagspace is a pair (T, p) where:*

- *T is a set of tags containing the distinguished root `_entity`,*
- *$p : T \setminus \{\texttt{_entity}\} \to T$ is the* parent function*, assigning each non-root tag its super-object.*

Every non-root tag reaches `_entity` after finitely many parent steps. There are no cycles (except the axiomatic self-reference of `_entity`, when p is extended to the root).

The tree structure of the example above:

```
_entity
  physical_object
    living_thing
      animal
        mammal
          dog
```

The tree expresses hierarchical knowledge: `dog` is related to `mammal`, which is related to `animal`, and so on. Each parent step contributes to the tag's structural identity.

Here `_is_a` establishes subordination. The distinct `_type_of` relation adds an inheritance source without changing position (Section 7).

3 Rank: Encoding the Tree in Strings

3.1 The Rank Function

Each tag has a rank encoding its structural position.

Definition 3.1 (Rank Function)**.** *The rank function $r : T \to \Sigma^*$ assigns to each tag a UTF-8 string such that r is injective and a child's rank extends its parent's rank by one code point. The root has the empty rank.*

For example, the following hard tags have these ranks. Dots separate the code points along a path; the stored rank is a byte string [10]:

Tag	Rank (dotted)	Position
`_entity`	(empty)	level 0, root
`_sub`	1	level 1
`_is_a`	1.1	level 2
`_type_of`	2	level 1
`_rel`	3	level 1
`_has`	3.1	level 2
`_true`	4	level 1
`_false`	5	level 1
`_interrogator`	6	level 1

3.2 The Prefix Partial Order

Definition 3.2 (Containership Relation)**.** *Define the* ***containership relation*** $\sqsubseteq$ *on* T *by:*

$$a \sqsubseteq b \iff r(a) \text{ is a prefix of } r(b).$$

That is, $a \sqsubseteq b$ *means* a *is an ancestor of* b *(or* $a = b$*). Tag* a contains *tag* b *in the hierarchy.*

3.3 Verifying the Partial Order Axioms

Theorem 3.1. *The containership relation* $\sqsubseteq$ *is a partial order on* T*.*

Proof. Prefix inclusion is reflexive and transitive. Mutual prefix inclusion gives equal strings, and injectivity of r then gives equal tags. □

Therefore $(T, \sqsubseteq)$ is a **poset** (partially ordered set).

3.4 Subtrees and Their Intersection

For $x \in T$, define the descendant subtree, including its root, by

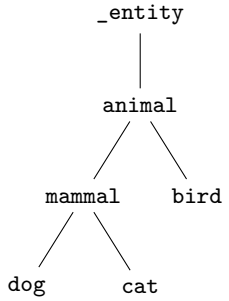
$$D(x) = \{t \in T : x \sqsubseteq t\}.$$

The unique-parent property makes this family *laminar*: two subtrees are nested or disjoint. Explicitly,

$$D(x) \cap D(y) = \begin{cases} D(y), & x \sqsubseteq y, \\ D(x), & y \sqsubseteq x, \\ \varnothing, & \text{otherwise.} \end{cases}$$

If a tag belonged to both subtrees, both roots would lie on its unique ancestor chain and hence would be comparable. This proves the last case; the first two follow directly from transitivity.

For example, with `dog` and `cat` subordinate to `mammal`, and `bird` on a separate branch subordinate to `animal`, $D(\texttt{animal}) \cap D(\texttt{mammal}) = D(\texttt{mammal})$, whereas $D(\texttt{mammal}) \cap D(\texttt{bird}) = \varnothing$. This is the set-theoretic basis for retaining the more specific root when intersecting subtree selections. A descendant subtree does not include its root's strict ancestors; shared ancestry therefore does not change the empty-intersection case. Common ground between entire tagspaces is considered in Section 5.3.



3.5 Lexicographic Total Order

Ranks also carry a total **lexicographic order**. Strings with a common prefix form a contiguous interval, so each structural subtree sorts together.

3.6 CS Tree versus Topological Poset: Orientation

A tree drawing places `_entity` at the top, but the ancestor-first prefix order makes it the **least element** ($\bot$): its empty rank prefixes every rank and has the lowest value under binary lexicographic collation using C++'s less-than operator `<`. This total order extends the prefix partial order; it also compares unrelated branches. Leaves are **maximal elements** of the partial order, not necessarily greatest in lexicographic order.

4 The Tagspace Topology

4.1 From Partial Order to Topology

We choose the upper-set Alexandrov topology of the ancestor-first poset $(T, \sqsubseteq)$ [6]. This makes the tagspace a topological space whose basic open sets are its subtrees:

Definition 4.1 (Tagspace Open Sets)**.** *A subset $U \subseteq T$ is **open** if it is upward-closed under $\sqsubseteq$:*

$$U \in \mathcal{T} \iff \forall\, a \in U,\ \forall\, b \in T :\ a \sqsubseteq b \Rightarrow b \in U.$$

In the tagspace tree, "upward-closed" means: if a tag is in the set, all its descendants are too. The smallest open neighbourhood of x is its subtree

$$N(x) = \{b \in T : x \sqsubseteq b\} = D(x) = \uparrow x.$$

Each $D(x)$ is a principal filter; the closure of $\{x\}$ is its ancestry, the principal ideal $\downarrow x = \{b \in T : b \sqsubseteq x\}$. There is also a dual Alexandrov topology of downward-closed sets, in which $D(x)$ is a closed set. Both topologies arise from the same tree; this paper keeps the upward-open convention unless explicitly stated otherwise.

This construction is known as the **Alexandrov topology** — the topology generated by a partial order in which arbitrary intersections (not just finite ones) of open sets remain open.

4.2 Verifying the Topology Axioms

Theorem 4.1. *Let T be the set of tags in a tagspace and let $\mathcal{T}$ be the collection of upward-closed subsets under $\sqsubseteq$. Then $(T, \mathcal{T})$ is a topological space.*

Proof. The empty and full sets are up-sets. Arbitrary unions and intersections of up-sets remain up-sets, giving both the topology axioms and the Alexandrov property. □

4.3 Specialization Order

Every topological space carries a **specialization preorder**. We use the convention $a \leq_{\text{sp}} b$ if $a \in \text{cl}(\{b\})$.

Proposition 4.1. *In the tagspace topology, $a \leq_{\text{sp}} b$ if and only if $a \sqsubseteq b$: the singleton closure consists of the tag and its ancestors.*

Proof. Every neighbourhood of a contains b iff its smallest neighbourhood $D(a)$ does, which is precisely $a \sqsubseteq b$. □

Thus `animal` $\leq_{\text{sp}}$ `mammal` $\leq_{\text{sp}}$ `dog`. Topological specialization in this convention runs opposite to taxonomic specialization: the closure of a specific tag contains its more general ancestors.

4.4 Continuity as Order-Preservation

Definition 4.2 (Tagspace Map)**.** *A map $f : T_1 \to T_2$ between two tagspaces is **topologically continuous** if inverse images of open sets are open: $f^{-1}(U) \in \mathcal{T}_1$ for every $U \in \mathcal{T}_2$.*

Proposition 4.2. *A map $f : T_1 \to T_2$ is continuous (with respect to the Alexandrov topologies) if and only if it is order-preserving: whenever $a \sqsubseteq b$ in T_1, we have $f(a) \sqsubseteq f(b)$ in T_2.*

Proof. A monotone map pulls up-sets back to up-sets, so it is continuous. Conversely, continuity makes $f^{-1}(D(f(a)))$ open and containing a. If $a \sqsubseteq b$, this preimage contains b, hence $f(a) \sqsubseteq f(b)$. □

This gives a precise mathematical definition of a **hierarchy-preserving map** between tagspaces. If `dog _is_a mammal` in T_1, a continuous map must send `dog` to a descendant of the image of `mammal` in T_2, possibly that image itself. Preserving labels, sub-relators, and predicates supplies additional conditions when a map is to preserve the full semantic structure.

5 The Hard Tag Base: A Common Subspace

5.1 Hard Tags

Tagspaces using the same hard-tag definitions contain a common hierarchy.

5.2 The Hard Tag Subspace

Definition 5.1 (Hard Tag Subspace)**.** *Let $H \subseteq T$ denote the set of hard tags. For tagspaces using the same hard-tag definitions, the induced subspace topology on H is the same, since its restricted ancestry order is the same. Thus H is a common subspace shared by these tagspaces.*

Remark 5.1. *One can define a category of extensions of H: objects are order embeddings $i_T : H \hookrightarrow T$, and morphisms are monotone maps $f : T \to T'$ satisfying $f \circ i_T = i_{T'}$. In this category, the object $H \xrightarrow{\mathrm{id}} H$ is **initial**: it has a unique morphism to each object.*

5.3 Interoperability via the Common Base

Each tagspace is a world of discourse. When these worlds disagree about user-defined tags, the same version of the hard-tag hierarchy H supplies common ground for exchanging and interpreting TAGL.

After identifying their copies of H, and any further tags whose identities and structure agree, two tagspaces satisfy

$$H \subseteq T_1 \cap T_2.$$

If they share no user-defined tags, then $T_1 \cap T_2 = H$. Such tagspaces need not be laminar: $T_1 = H \cup \{a\}$ and $T_2 = H \cup \{b\}$, with distinct user tags a, b, overlap without either containing the other. Laminarity applies to descendant subtrees within one tree. Equal labels alone do not establish agreement between independently defined user tags.

For extensions in which H contains every ancestor of its tags, gluing only the copies of H gives a **categorical pushout** $T_1 \amalg_H T_2$ in posets with monotone maps. Keep the user tags distinct and inherit each tree's parent relation. The result is again a rooted tree: parents agree on H, and every other tag retains its original parent. Any two monotone maps agreeing on H extend uniquely to this union, which is the pushout's universal property. The Alexandrov functor of Section 9.1, being a left adjoint, carries this pushout to topological spaces. This supplies a precise model for combining worlds; reconciling user labels, predicates, and rank assignments remains part of the merge design.

```
-- shared by tagspaces using the same hard-tag version:
_entity _sub _entity

-- user-defined, varies by tagspace:
>> dog _is_a mammal;
```

Listing 1: A shared hard-tag version; distinct user extensions

6 Predicates: Fibered Structure

6.1 Relators Are Tags

Predicate relators are themselves tags — subordinate to `_rel` in the tagspace tree. This means predicates are not outside the system; they are *inside* it. The tagspace is self-describing.

6.2 Predicates as Horizontal Relations

Predicates are relations attached to a tag that do not alter its position in the tree. TAGL groups objects under a relator; each attached predicate records one relator, one object, and optional modifier information [7, 3].

```
>> dog _is_a mammal     -- vertical: places dog in the tree
    _has legs = 4, tail
    _can bark;          -- horizontal: predicate data at dog
```

Listing 2: Vertical identity and horizontal predicates

6.3 Declared Predicate Fibers

The structural base and declared predicates have the following decomposition. It describes the predicate data within the integrated model; the additional type dependencies are given in Section 7:

Definition 6.1 (Structural Base and Predicate Fibers)**.** *1.* ***Base space****: the topological space $(T, \mathcal{T})$ defined by subordinate relations and ranks — the vertical dimension.*

2. ***Fiber****: at each tag $t \in T$, a set of predicates $P(t) \subseteq \mathrm{Rel} \times T \times \mathrm{Constraint}$ — the horizontal dimension, where* Rel *is the set of relator tags and* Constraint *records a comparison and typed modifier, or a bare assertion.* Mod *denotes the supported semantic value domain.*

Remark 6.1 (Predicate fibers)**.** *Let $P(t)$ be the predicate set attached to tag $t \in T$. Form the disjoint union of these sets and project each tagged predicate onto its tag:*

$$E = \coprod_{t \in T} P(t) = \{(t, q) \mid t \in T,\ q \in P(t)\}, \qquad \pi : E \to T, \quad \pi(t, q) = t.$$

The fiber over t is

$$E_t = \pi^{-1}(\{t\}) = \{(t, q) \mid q \in P(t)\} \cong P(t).$$

Thus each predicate set is a ***fiber*** *over its tag, in the precise set-theoretic sense.*

The projection π brings the two layers together: the base space records hierarchical position, and the fiber records the predicates attached there.

This suggests a connection with **fiber bundles**. A topological bundle structure requires a topology on E, continuity of π, and local trivializations $\pi^{-1}(U) \cong U \times F$ respecting projection. Ordering predicates by rank can support indexing; establishing these bundle conditions is a separate mathematical construction.

7 Inheritance

Each tag s explicitly declares type objects $\tau(s) \subseteq T$ through modifier-free type relations. Draw $s \to p(s)$ for $s \neq \bot$ and $s \to y$ for each stored type object $y \in \tau(s)$. Require this finite dependency graph to be acyclic. Ordinary predicate references remain data.

Definition 7.1 (Inheritance order)**.** *$x \preceq s$ iff $s \to^* x$. The inheritors of x are the principal up-set $I(x) = \{s : x \preceq s\}$.*

Proposition 7.1. *$\preceq$ is a partial order with least element $\bot$, and $\sqsubseteq \subseteq \preceq$. Its Alexandrov topology satisfies $\mathcal{T}_{\preceq} \subseteq \mathcal{T}$.*

Proof. Reflexivity and transitivity hold by construction; $x \preceq s \preceq x$ with $x \neq s$ would give a cycle. Every tag reaches $\bot$ through parents, and $x \sqsubseteq s$ places x on that chain. A $\preceq$-up-set is therefore a $\sqsubseteq$-up-set. □

Ranks encode $\sqsubseteq$; type relations extend the inheritance order without changing ranks or identifying subjects.

Definition 7.2 (Predicate specialization and normalization)**.** *Write $q \triangleright p$ when p is bare, both its relator and object are structural ancestors of the corresponding components of q, and at least one component is strictly more specific or q supplies a value. For a finite set S, define*

$$\nu(S) = \{p \in S : \nexists q \in S\ (q \triangleright p)\}, \qquad \bar{P}(s) = \nu\left(\bigcup_{x \preceq s} P(x)\right).$$

$\bar{P}(s)$ is the set of effective predicates. Set union includes a shared source once, independently of the number of paths to it. Explicit declarations and distinct typed valued contributions are preserved at their sources.

Definition 7.3 (Consistency and valued satisfaction)**.** *For an exact relator–object pair k, let $C_k(x) \subseteq \mathrm{Mod}$ be the values admitted by x's supplied constraints on k, or* Mod *if none are supplied. Define*

$$A_k(s) = \bigcap_{x \preceq s} C_k(x).$$

A state is consistent when every such intersection has a representable witness. Let $V_k(s)$ mean that some source $x \preceq s$ supplies a modifier on k. A valued query admitting $Q \subseteq \mathrm{Mod}$ holds at s iff $V_k(s)$ and $\varnothing \neq A_k(s) \subseteq Q$.

The presence condition excludes missing and wholly unspecified values. Bare predicates impose no value constraint. Values on different exact pairs remain independent, and numeric and textual constraints do not coerce into one another. For example, $2 < v < 5$ admits 3 but does not entail $v = 3$. Consistency is checked on all contributions before normalization.

Proposition 7.2 (Persistence)**.** *If $x \preceq s$, then $A_k(s) \subseteq A_k(x)$. In a consistent state, tags satisfying a valued query form a $\preceq$-open set.*

Proof. Transitivity gives $\{y : y \preceq x\} \subseteq \{y : y \preceq s\}$, so the intersection for s includes every constraint for x. The supplied-value witness at x is also a source of s. Thus $V_k(x)$ and $A_k(x) \subseteq Q$ imply $V_k(s)$ and $\varnothing \neq A_k(s) \subseteq Q$. □

Persistence concerns satisfaction within a fixed state. Normalization can replace a bare general predicate by a more specific one while preserving that satisfaction. Mutation validation maintains consistency between states; its operational contract is specified by TAGL [7].

7.1 Query Semantics as Selection of Sets

A structural selector without predicates returns the immediate children of x. With a predicate condition, it selects within the subtree $D(x)$ under $\sqsubseteq$. Positive predicate conditions inspect $\bar{P}(t)$ under $\preceq$. Unrestricted effective positive predicates give inheritance-open sets. Structural filters, full-text searches, and arbitrary state-inspection queries need not give inheritance-open results.

```
?? _what _sub mammal;           -- immediate structural children
?? _what _has blood;            -- mammal and its inheritors
?? _what can bark;              -- dog and its inheritors
?? _what _has blood can bark;   -- subjects satisfying both effectively
```

Listing 3: Structure and effective predicate selection

Here `can` is the example's user-defined relator. An inherited blood assertion and a local bark assertion can jointly establish the final query.

8 Structural Identity

8.1 Identity Is Position

The TAGL specification distinguishes nominal identity (the label) from structural identity (the subordinate relation and rank) [7].

In tagd, meaning is defined through relations. The string `dog` provides a name; its relations to `mammal`, `animal`, and ultimately `_entity` establish its place in the tagspace. Its predicates express how it relates to other tags.

Moving the tag changes its structural identity while its nominal label can remain the same. This distinction makes the maxim precise: *to be, is to be related.* A label locates the tag; its position and relations express what it means within the system.

8.2 The Canonical Form

Canonical serialization expresses explicit declarations as TAGL text [8].

The defining requirement for canonical serialization is the round-trip law

$$\text{load}(\text{dump}(T)) \cong T,$$

where the isomorphism preserves labels, subordinate relations, type declarations, and predicate values with their operator and type metadata. The text then represents the same relational structure in a form that can be stored, exchanged, and reconstructed.

Cryptographic versioning additionally requires deterministic encoding (Section 10.4).

9 Deeper Structures

The preceding sections connect tags, ranks, and predicates to familiar mathematical structures. These connections extend naturally into category theory and logic, providing a vocabulary for reasoning about the system and developing it further. Category theory, introduced by Samuel Eilenberg and Saunders Mac Lane [16], supplies the language of functors, adjunctions, and universal constructions used here.

9.1 The Adjunction Between Order and Topology

The relationship between the tagspace as a partial order and the tagspace as a topological space is not incidental. It is a formal **adjunction** — one of the most fundamental relationships in category theory.

Two functors mediate between the two views:

- $A : \mathbf{PreOrd} \to \mathbf{Top}$ (the *Alexandrov functor*) takes the rank prefix partial order on tags and produces the Alexandrov topological space described in Section 4.
- $S : \mathbf{Top} \to \mathbf{PreOrd}$ (the *specialization functor*) takes any topological space and recovers a preorder: $a \leq b$ iff every open set containing a also contains b.

$$\mathbf{PreOrd} \underset{S}{\overset{A}{\underset{\longleftarrow}{\overset{\longrightarrow}{\bot}}}} \mathbf{Top}$$

These functors form an adjunction: A is left adjoint to S, written $A \dashv S$. Applying both in sequence recovers the original structure: $S(A(P)) \cong P$.

The unit $P \to S(A(P))$ is the identity on underlying points: passing from the order to its topology and back recovers the same order. For tagd, this explains why rank-prefix containment and the neighbourhood structure are two descriptions of the same hierarchy.

9.2 Topological Properties of the Tagspace

The Alexandrov topology gives the tagspace several useful local properties.

First countable. Every tag x has a smallest open neighbourhood: the set $N(x)$ of descendants of x, including x itself. The single set $N(x)$ forms a local base at x.

Locally compact. For any tag x, the smallest open neighbourhood $N(x)$ is compact in the Alexandrov topology. Any open cover of $N(x)$ must include a member containing x, which then contains all of $N(x)$, giving a one-member subcover. This proves compactness of the smallest open neighbourhood, without a Hausdorff assumption.

Locally path connected. For $y \in N(x)$, the map $\gamma : [0,1] \to N(x)$ with $\gamma(0) = x$ and $\gamma(t) = y$ for $t > 0$ is continuous: the inverse image of an open set is empty, $(0,1]$, or $[0,1]$, each open in $[0,1]$. Hence each $N(x)$ is path connected, so these smallest neighbourhoods give local path connectedness.

Subspaces and finite products are Alexandrov. Any subset of tags — for example, the result of a `query()` or all descendants of a given tag — inherits an Alexandrov subspace topology. Subsets can therefore be studied with the order and topology restricted to their own tags.

9.3 Modal Operators: Interior and Closure

Use the inheritance topology for effective propositions. Its Kripke accessibility is $xRy \iff x \preceq y$. For any valuation $V \subseteq T$,

$$\Box V = \{x : I(x) \subseteq V\} = \operatorname{int}(V), \qquad \Diamond V = \{x : I(x) \cap V \neq \varnothing\} = \operatorname{cl}(V).$$

Necessity checks every inheritor, including the subject; possibility has an inheritor witnessing the valuation. The structural topology has analogous operators using $D(x)$.

Proposition 9.1 (S4 frame)**.** *The inheritance frame validates $\Box P \to P$ and $\Box P \to \Box\Box P$.*

Proof. Reflexivity gives $x \in I(x)$. Transitivity gives $y \in I(x) \Rightarrow I(y) \subseteq I(x)$, so a valuation true throughout $I(x)$ is necessarily true at every $y \in I(x)$. □

The interior algebra on $\mathcal{P}(T)$ supplies the usual S4 semantics [15]. More specific frames can validate additional formulas.

Proposition 9.2 (Possibility and branching)**.** *$\Diamond$ maps every inheritance-open set to an open set iff each principal up-set $I(x)$ is directed: any $y, z \in I(x)$ have a common upper bound.*

Proof. Suppose $x \in \Diamond P$ for open P, witnessed by $z \in I(x) \cap P$. For $y \in I(x)$, directedness supplies $w \succeq y, z$. Openness gives $w \in P$, hence $y \in \Diamond P$. Conversely, given $y, z \in I(x)$, apply preservation to $P = I(z)$. Since $x \in \Diamond I(z)$, openness gives $y \in \Diamond I(z)$, providing a common upper bound of y and z. □

At a fork without a common extension, a possible open proposition can therefore cease to be possible along the other branch.

Proposition 9.3 (Double negation)**.** *For an open proposition P, $\Box P = P$ and $\Box\Diamond P = \neg\neg P$ in the open-set Heyting algebra.*

Proof. An open set equals its interior. Intuitionistic negation is $\neg P = \operatorname{int}(T \setminus P)$, while $\operatorname{cl} P = T \setminus \operatorname{int}(T \setminus P)$. Taking interiors gives the second identity. □

These equalities concern sets of satisfying worlds. Constructive evidence for a proposition is the separate dependent-type structure below.

9.4 Constructive Existence: Tags as Witnesses

In **constructive mathematics**, existence is established by supplying a witness and evidence for its defining property. The **Brouwer–Heyting–Kolmogorov (BHK) interpretation**, named for L. E. J. Brouwer, Arend Heyting, and Andrey Kolmogorov, gives this reading to intuitionistic logic [13]. The **Curry–Howard correspondence**, associated with Haskell B. Curry and William A. Howard, expresses propositions as types and proofs as terms (programs). Per Martin-Löf's **intuitionistic type theory (MLTT)** makes the existential reading explicit through a dependent sum, or Σ-type [14]:

$$t : T, \quad e : Q(t) \quad \Longrightarrow \quad (t, e) : \sum_{x:T} Q(x).$$

Here $Q(x)$ is the type of evidence for a specified property of x. The tag t is the **witness**; the pair (t, e) is the constructed proof of $\exists x : T.\, Q(x)$.

For example, let $Q(x)$ express that x has the recorded subordinate relation `_is_a mammal`. Checking that relation for `dog` supplies evidence for this structural claim, making `dog` its existential witness. The evidence establishes the relation within the tagspace; further interpretations of `_is_a` require their own rules.

A proof-carrying formalization can make this evidence explicit as a term $e : Q(t)$ and certify the tagspace invariants. Its typing and validation rules would connect TAGL constructions to the established BHK and Curry–Howard interpretations (Section 10.2).

10 Directions for Formalization

10.1 Kripke Semantics and Modal Queries

A modal evaluator would implement the operators of Section 9.3 over $I(x)$. Its proof obligations include preserving the distinction between arbitrary state tests and persistent open propositions.

10.2 Martin-Löf Type Theory and Dependent Structure

The indexed family $P(t)$ gives a set-theoretic dependent sum $\sum_{t:T} P(t)$. Evidence types $Q(t)$ instead give proofs (t, e) with $e : Q(t)$, and a dependent function $f : \prod_{t:T} Q(t)$ provides evidence $f(t) : Q(t)$ for every tag [14]. A verified evaluator needs explicit typing, equality, and total computation rules for these evidence types.

10.3 Computation and Processes

Functions represented as tags and message-based processes are proposed in the TAGL specification and design thesis [7, 8]. Order-preserving computations would be continuous maps of the chosen tagspace orders. A runtime must establish the corresponding typing and evaluation invariants.

10.4 Signed Merkle Versions of Tagspaces

Canonical declarations can support a Merkle commitment by structural recursion from leaves to root [11]. The encoding must cover labels, ranks, subordinate relations, explicit type relations, and complete predicate metadata. A signed manifest can authenticate the encoding version and separate commitments to the hard-tag base and full tagspace [12]; identity of digests then depends on collision resistance and deterministic encoding.

A Implementation Correspondences

Mathematical structure	Code symbol or representation
Structural parent and identity	`abstract_tag::super_object`, `sub_relator`
Prefix order $\sqsubseteq$	`rank`; empty root handled by tagspace containment
Type declarations $\tau(s)$	`type_predicate_set`, `abstract_tag::type_relations`
Explicit predicates $P(s)$	`predicate_set`, `abstract_tag::relations`
Inheritance order $\preceq$	Reachability through structural parents and type objects
Normalization ν	`normalize_predicates`
Effective predicates $\bar{P}(s)$	`tagdb::relation_resolver`
Canonical serialization	TAGL `dump`; explicit declarations and typed modifiers

The language and storage contracts are specified in TAGL [7]; source definitions are maintained in the corresponding modules [1, 2, 3, 4, 5].

References

[1] tagd source code, `tagd/include/tagd/hard-tags.h`. Hard tag definitions and axioms.

[2] tagd source code, `tagd/include/tagd/rank.h`. Rank structure, prefix order, and `contains()` implementation.

[3] tagd source code, `tagd/include/tagd.h`. `abstract_tag`, `predicate_set`, and tag set operations.

[4] tagd source code, `tagd/src/effective-predicate.cc` and `tagspace/tagdb/src/relation-resolver.cc`. Predicate specialization, normalization, and effective satisfaction.

[5] tagd source code, `tagspace/tagdb/sqlite/src/sqlite.cc`. Schema, rank allocation, queries, mutation, and serialization.

[6] Mathlib, *Upper and lower sets topologies*. `https://leanprover-community.github.io/mathlib4_docs/Mathlib/Topology/Order/UpperLowerSetTopology.html`. Upper-set topology, singleton closure, and monotone continuous maps.

[7] *TAGL Specification*, `docs/TAGL-spec.md`. Language specification and semantic model.

[8] *tagd Design Thesis*, `docs/tagd-thesis.md`. Language design document including function model and computation semantics.

[9] *tagd README*, `README.md`. Working examples and TAGL tutorial.

[10] *Example tagspace and rank listing*. The supplied animal hierarchy is defined in `tagsh/tests/bootstrap.tagl`. Its `.dump_grid` listing shows each tag's parent and rank. Generate it with:

```
cd tagsh/
bin/tagsh --file tests/bootstrap.tagl --tagl '.dump_grid' --noshell
```

[11] B. Laurie et al., *Certificate Transparency Version 2.0*, RFC 9162, Section 2.1 (Merkle trees). `https://www.rfc-editor.org/rfc/rfc9162.html`.

[12] NIST, *Digital Signature Standard (DSS)*, FIPS 186-5, 2023. `https://doi.org/10.6028/NIST.FIPS.186-5`.

[13] Philip Wadler, *Propositions as Types*. Communications of the ACM 58(12), 75–84, 2015. `https://doi.org/10.1145/2699407`. BHK interpretation and the Curry–Howard correspondence.

[14] Per Martin-Löf, *Intuitionistic Type Theory*. Notes by Giovanni Sambin, Bibliopolis, 1984. `https://archive-pml.github.io/martin-lof/pdfs/Bibliopolis-Book-retypeset-1984.pdf`. Dependent sums, dependent products, and their logical interpretations.

[15] André Platzer, *Classical Modal Logic*. 15-816 lecture notes, Carnegie Mellon University, 2010. `https://www.cs.cmu.edu/~fp/courses/15816-s10/lectures/05-pml.pdf`. S4 axiomatics and historical references to Lewis and Kripke.

[16] Samuel Eilenberg and Saunders Mac Lane, *General Theory of Natural Equivalences*. Transactions of the American Mathematical Society 58(2), 231–294, 1945. `https://doi.org/10.1090/S0002-9947-1945-0013131-6`.